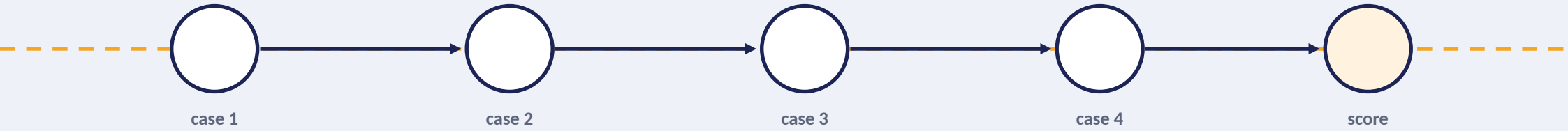
LANGGRAPH · CHAPTER 14 · FLASHCARDS

Evaluation



Golden sets, judges and gates

13 code cards on scoring an agent before and after it ships



Target, evaluator, evaluate()

A target turns inputs into outputs; an evaluator scores them against the reference

- LangSmith hands each evaluator its arguments by name: inputs, outputs, reference_outputs.
- Return no "key" and the score is named after the function, here exact_match.

PYTHON

```python
from langsmith import Client, evaluate
from langsmith.schemas import Example
from uuid import uuid4

def target(inputs: dict) -> dict:                # the system under test
    return {"answer": inputs["text"].lower()}    # forgets to trim, on purpose

def exact_match(outputs: dict, reference_outputs: dict) -> dict:
    matches = outputs["answer"] == reference_outputs["answer"]
    return {"score": int(matches),
            "comment": "matches" if matches else "text differs"}

examples = [
    Example(id=uuid4(), inputs={"text": "Stanley Bridge"},
            outputs={"answer": "stanley bridge"}),
    Example(id=uuid4(), inputs={"text": "  Montaza Gate  "},
            outputs={"answer": "montaza gate"}),
]
results = evaluate(target, data=examples, evaluators=[exact_match],
                   client=Client(auto_batch_tracing=False),
                   upload_results=False)         # local: no keys, no upload
# 'Stanley Bridge': exact_match=1
# '  Montaza Gate  ': exact_match=0
```

A scorer behind a wrapper

The scorer is plain Python; the wrapper adapts it to LangSmith's dicts

- The target never sees the expected keywords: only the evaluator gets reference_outputs.
- The scorer knows nothing about LangSmith, so a unit test can call it directly.

PYTHON

```python
# The system under test: one case's inputs in, a dict of outputs out
def target(inputs: dict) -> dict:
    reply = assistant.invoke([
        ("system", f"You answer ... using only this timetable:\n{TIMETABLE}"),
        ("human", inputs["question"]),
    ])
    return {"answer": reply.content}

# The scorer: share of expected keywords found, and which are missing
def keyword_coverage(answer: str,
                     expected_keywords: list[str]) -> tuple[float, list[str]]:
    missing = [k for k in expected_keywords if k.lower() not in answer.lower()]
    return 1 - len(missing) / len(expected_keywords), missing

# The wrapper: pick the fields, call the scorer, package the feedback
def keyword_coverage_eval(outputs: dict, reference_outputs: dict) -> dict:
    score, missing = keyword_coverage(outputs["answer"],
                                      reference_outputs["expected_keywords"])
    return {"key": "keyword_coverage", "score": score,
            "comment": f"missing {missing}" if missing else "all keywords present"}
```

Upload once, then evaluate

The golden cases become a LangSmith dataset; each evaluate() run is an experiment

- has_dataset() skips the upload, so later edits to GOLDEN_CASES do not reach LangSmith.
- The experiment stores a row per case: the answer, its trace, the score and the comment.

PYTHON

```python
DATASET = "travel-assistant-golden"

def ensure_dataset(client: Client) -> None:
    if client.has_dataset(dataset_name=DATASET):
        return                  # from now on the LangSmith copy counts
    dataset = client.create_dataset(DATASET, description="Five golden cases ...")
    client.create_examples(dataset_id=dataset.id, examples=[
        {"inputs": {"question": question},
         "outputs": {"expected_keywords": keywords}}
        for question, keywords in GOLDEN_CASES
    ])

client = Client()
ensure_dataset(client)
results = client.evaluate(target, data=DATASET, evaluators=[keyword_coverage_eval],
                          experiment_prefix="travel-assistant")
for row in results:
    result = row["evaluation_results"]["results"][0]
    print(f"{result.score:.2f}  {row['example'].inputs['question']}",
          result.comment)
# 1.00  When does flight EK924 leave Cairo?  all keywords present
# 1.00  What time is the London flight on a Tuesday?  all keywords present
```

Word overlap versus meaning

Heuristics reward wording: a wrong answer in the reference's words outscores a right one

- Rule-based checks are exact and free; heuristics give a smooth number; a judge reads meaning.
- Here only the keyword check separates the two, and only because "Cairo" is a keyword.

PYTHON

```python
REFERENCE = "Flight EK924 leaves Cairo at 10:15."
KEYWORDS = ["EK924", "Cairo", "10:15"]
ANSWERS = {"A (right)": "At 10:15, EK924 departs from Cairo.",
           "B (wrong)": "Flight EK924 leaves Dubai at 10:15."}

def words(text: str) -> set[str]:
    return set(re.findall(r"\w+", text.lower()))

# Heuristic, BLEU-1 style: how much of the answer is also in the reference
def word_precision(answer: str, reference: str) -> float:
    return len(words(answer) & words(reference)) / len(words(answer))

# Heuristic, ROUGE-1 style: how much of the reference the answer covers
def word_recall(answer: str, reference: str) -> float:
    return len(words(answer) & words(reference)) / len(words(reference))

# $ python 12-Evaluation/02-metric-types/rule_and_heuristic.py
# answer      exact  keywords  precision  recall
# A (right)   0.0    1.00      0.71       0.71
# B (wrong)   0.0    0.67      0.86       0.86     <- wrong, yet higher overlap
```

Code metrics that say why

Each metric returns a MetricResult: a name, a 0-1 score and a note explaining it

- tool_call_accuracy reads the FIRST tool, so a skipped search fails the case.
- Retrieval metrics grade the files the agent's own searches returned, not a search you run.

PYTHON

```python
# Rule-based: 1.0 if the first tool the agent called is the expected one
def tool_call_accuracy(actual_tool: str | None,
                       expected_tool: str | None) -> MetricResult:
    hit = actual_tool is not None and actual_tool == expected_tool
    return MetricResult("tool_call_accuracy", 1.0 if hit else 0.0,
                        f"called {actual_tool or 'no tool'}, "
                        f"expected {expected_tool or 'no tool'}")

# Retrieval: share of the expected files retrieved at least once
def context_recall(retrieved: Sequence[str],
                   relevant: Sequence[str]) -> MetricResult:
    if not relevant:
        return MetricResult("context_recall", 1.0, "no sources required")
    missing = [source for source in relevant if source not in retrieved]
    score = (len(relevant) - len(missing)) / len(relevant)
    return MetricResult("context_recall", round(score, 3),
                        f"missing {missing}" if missing else "all sources retrieved")

# tool_call_accuracy(None, "get_weather") -> 0.0  "called no tool, expected ..."
# context_precision(["a", "a", "b", "c"], ["a"]) -> 0.5  2/4 chunks relevant
```

Metrics wrapped as evaluators

Each wrapper picks fields, calls one metric, and returns a key, a score and a comment

- An evaluator asks only for the arguments it reads: length_penalty_eval takes outputs alone.
- Keeping metrics apart lets the online scorer reuse them and pytest test them with no keys.

PYTHON

```python
# A MetricResult becomes the feedback LangSmith stores on the case's trace
def _feedback(result: MetricResult) -> dict:
    return {"key": result.name, "score": result.score, "comment": result.note}

def keyword_coverage_eval(outputs: dict, reference_outputs: dict) -> dict:
    return _feedback(keyword_coverage(outputs["answer"],
                                      reference_outputs["expected_keywords"]))

def tool_call_accuracy_eval(outputs: dict, reference_outputs: dict) -> dict:
    return _feedback(tool_call_accuracy(outputs["tool"],
                                        reference_outputs["expected_tool"]))

def length_penalty_eval(outputs: dict) -> dict:      # no reference needed
    return _feedback(length_penalty(outputs["answer"]))

ALL_EVALUATORS = [keyword_coverage_eval, tool_call_accuracy_eval, rouge_eval,
                  length_penalty_eval, context_precision_eval, context_recall_eval,
                  toxicity_eval, correctness_eval, relevance_eval]

# $ python -m pytest -q 12-Evaluation/04-agentic-rag-evaluation/tests
# 14 passed
```

A judge with a fixed rubric

The rubric prices each kind of error; the schema puts the reason before the score

- The steps stay fixed for every case; each case's expected_answer says what right means.
- A gpt-4o judge at temperature 0 grades the gpt-4o-mini agent; 0-10 is scaled to 0-1.

PYTHON

```python
CORRECTNESS_STEPS = """...
1. List each fact in the expected answer and check that the answer states it.
2. List each fact the answer states that the expected answer doesn't. ...
Start at 10. Subtract at least 3 for each missing expected fact and at least 5 for
each wrong fact. Name every missing and every wrong fact in your reason."""

class Grade(BaseModel):            # reason first: the model explains, then scores
    reason: str = Field(description="One or two sentences explaining the grade.")
    score: int = Field(ge=0, le=10)

@cache                             # build the judge once, on first use
def _judge():
    return ChatOpenAI(model=config.JUDGE_MODEL,
                      temperature=0).with_structured_output(Grade)

def _grade(name: str, steps: str, prompt: str) -> MetricResult:
    grade = _judge().invoke([("system", steps), ("human", prompt)])
    return MetricResult(name, grade.score / 10, grade.reason)

def correctness_judge(question: str, answer: str, expected_answer: str):
    return _grade("correctness", CORRECTNESS_STEPS,
                  f"Question:\n{question}\n\nExpected answer:\n{expected_answer}"
                  f"\n\nAnswer to grade:\n{answer}")
```

A case for every metric

Every field in a golden case exists because some metric reads it

- Each case is stored split three ways: inputs for the target, outputs for evaluators, metadata.
- After the first upload, the LangSmith dataset and its versions are the source of truth.

PYTHON

```python
# One case in evals/data/golden_dataset.json, and who reads each field:
#   "query": "Who is the sales rep for the Snowflake Platform opportunity?"
#   "expected_answer": "... is Sarah Lee."                    <- judge, rouge
#   "expected_keywords": ["Snowflake Platform", "Sarah Lee"]   <- keyword_coverage
#   "expected_tool": "search_knowledge_base"                  <- tool_call_accuracy
#   "expected_sources": ["crm_exports/opportunities.csv",     <- context_precision
#                        "crm_exports/sales_reps.csv"]           and context_recall

def ensure_dataset() -> None:
    if client.has_dataset(dataset_name=DATASET):
        return
    cases = json.loads(GOLDEN_DATASET_PATH.read_text(encoding="utf-8"))["cases"]
    dataset = client.create_dataset(DATASET, description="...")
    client.create_examples(dataset_id=dataset.id, examples=[{
        "inputs": {"query": case["query"]},
        "outputs": {k: case[k] for k in ("expected_answer", "expected_keywords",
                                         "expected_tool", "expected_sources")},
        "metadata": {"id": case["id"], "category": case["category"]},
    } for case in cases])
# created dataset company-kb-golden with 18 examples       (first run only)
```

The agent as the target

The target calls the same ask_with_tools the chat UI uses, with online scoring off

- ask_with_tools returns an AgentReply: answer, tools, sources, contexts and run_id.
- Metadata on the experiment records which agent and judge models produced the scores.

PYTHON

```python
# The system under test: one golden question in, what the agent did out
def target(inputs: dict) -> dict:
    reply = ask_with_tools(inputs["query"], online_eval=False)
    return {"answer": reply.answer,
            "tool": reply.tools[0] if reply.tools else None,   # the FIRST tool
            "tools": reply.tools,
            "sources": reply.sources, "contexts": reply.contexts}

def run(set_baseline: bool) -> int:
    run_ingestion()                  # 1. Chroma holds the company files
    ensure_dataset()                 # 2. upload the golden set on the first run
    results = client.evaluate(       # 3. every case through target + 9 evaluators
        target, data=DATASET, evaluators=ALL_EVALUATORS,
        experiment_prefix="kb-agent",
        metadata={"agent_model": config.AGENT_MODEL,
                  "judge_model": config.JUDGE_MODEL},
        max_concurrency=4,
    )
    ...                              # 4. pass rule and summary, 5. latest.json

# FAIL  snowflake_sales_rep   correctness 0.0  tool search_knowledge_base
# Pass rate: 94% (17/18)
```

GATE · REGRESSION

Pass rule and regression gate

A run fails on a low pass rate OR on any case that passed in the reviewed baseline

- Only two of the nine scores decide a case; the other seven are diagnostics.
- A pass rate can hold while one case breaks, so the gate also compares case by case.

PYTHON

```python
PASS_THRESHOLD = 0.8   # minimum correctness score for a case to pass
MIN_PASS_RATE = 0.8    # minimum share of passing cases for the run to pass

# A case passes when the judge rates it correct and the right tool came first
def case_passed(scores: dict) -> bool:
    return (scores["correctness"] >= PASS_THRESHOLD
            and scores["tool_call_accuracy"] == 1.0)

# Ids that passed in the baseline and fail (or are missing) now
def find_regressions(baseline_cases: list[dict],
                     current_cases: list[dict]) -> list[str]:
    current = {case["id"]: case["passed"] for case in current_cases}
    return [case["id"] for case in baseline_cases
            if case["passed"] and not current.get(case["id"], False)]

# Search tool's k changed from 4 to 2, then:
# $ python evals/run_eval.py; echo "exit code: $?"
# Pass rate: 94% (17/18)                <- the 80% rule alone would ship it
# REGRESSED graphrag_project_owner: passed in the baseline, fails now
# exit code: 1                          <- CI fails the step
```

Scores on the live trace

Live traffic has no golden answer, so only reference-free checks run there

- The run id is chosen before the agent runs, so late feedback still lands on its trace.
- Scoring runs in a background thread: the user gets the answer without waiting for it.

PYTHON

```python
def _score_online(run_id: str, question: str, answer: str) -> None:
    wait_for_all_tracers()      # the run must reach LangSmith before feedback
    client = Client()
    for result in (toxicity_score(answer), bias_fairness_score(answer)):
        client.create_feedback(run_id, key=result.name, score=result.score,
                               comment=result.note)
    try:
        grade = relevance_judge(question, answer)       # reference-free judge
        client.create_feedback(run_id, key="answer_relevancy",
                               score=grade.score, comment=grade.note)
    except Exception as exc:    # a failed judge call must not hide the others
        client.create_feedback(run_id, key="answer_relevancy_error",
                               comment=str(exc))

# Inside ask_with_tools:
run_id = uuid.uuid4()           # chosen up front so feedback can find the run
result = get_agent().invoke({"messages": messages},
                            config={"run_id": run_id, "run_name": "chatbot"})
thread = threading.Thread(target=_score_online,
                          args=(reply.run_id, question, reply.answer), daemon=True)
thread.start()
```

HUMAN REVIEW

From a flagged run to a golden case

Queue every low score plus a random sample; a reviewer writes the new expectation

- The sample matters: every online score was 1.0 on the answer that ranked colleagues.
- The expected answer is the reviewer's, never the production answer it replaces.

PYTHON

```python
# Every run with a low online score, plus a seeded random sample of the rest
def select_for_review(runs, flag_below=FLAG_BELOW, sample_size=SAMPLE_SIZE,
                      seed=14):
    flagged, unflagged = [], []
    for run in runs:
        low = [key for key in ONLINE_KEYS
               if run["scores"].get(key, 1.0) < flag_below]
        (flagged if low else unflagged).append(
            (run, f"flagged: {', '.join(low)}" if low else "sampled"))
    sampled = random.Random(seed).sample(unflagged, min(sample_size, len(unflagged)))
    return flagged + sampled
# sampled    Which sales rep is the worst performer?

# promote_case.py: the reviewed case joins the dataset, linked to its live run
client.create_example(
    inputs={"query": QUESTION},
    outputs=REVIEWED,              # written by the reviewer
    dataset_name=DATASET,
    metadata={"id": "rep_performance", "category": "policy",
              "tags": ["from-production"]},
    source_run_id=source.id,       # the example links back to the production trace
)
```

PITFALLS

Three traps in the harness code

A no-tool case can't pass, keywords match inside words, repetitions skew the gate

- For a "don't act" case, score actual_tool == expected_tool, so None vs None passes.
- Before the gate reads num_repetitions runs, fold the rows per id (say, 2 of 3 must pass).

PYTHON

```python
# 1. The chatbot's tool_call_accuracy fails a correct "don't act" case:
tool_call_accuracy(None, None)
# MetricResult(score=0.0, note='called no tool, expected no tool')
#    (02-metric-types' version scores the same call 1.0)

# 2. keyword_coverage matches substrings, the bug 14.6 fixed for safety:
keyword_coverage("We won't say", ["won"]).score       # 1.0

# 3. evaluate(..., num_repetitions=3) yields 3 rows per example (54, not 18),
#    run() counts each row as a case, and find_regressions keeps only the LAST:
find_regressions([{"id": "x", "passed": True}],
                 [{"id": "x", "passed": False},
                  {"id": "x", "passed": False},
                  {"id": "x", "passed": True}])        # [] : 1 of 3 still passes

# Checked locally on langsmith 0.9.8: no model call, no upload
```